Failure Is Not a Score: Auditing Defense Utility Collapse and Infrastructure Provenance in LLM-Agent Benchmarks

Md Sharif Hossain and Faez Mahmud

***Abstract*—Benchmark scores for tool-using large language model (LLM) agents are widely presumed to reflect intrinsic model capability and robustness. However, current evaluation pipelines conflate two distinct sources of measurement distortion: *behavioral distortion*, where security defenses achieve low attack success only by destroying legitimate agent utility, and *infrastructure distortion*, where unresolved runtime or provider faults are silently recorded as model task failures. We present a dual-layer empirical audit quantifying both distortions across six agent benchmark harnesses and twelve production model configurations (20B to 550B total parameters). First, across 2,636 planned executions in AgentDojo (2,310 verified completions across Groq and OpenRouter cohorts), we compare capability-restriction (Tool Filter) against content-delimiting defenses (Spotlighting, Repeat Prompt). While Tool Filter drives targeted attack success to 0.0% on open-weight backbones, it induces catastrophic benign utility collapse—dropping paired task completion by 84.6 percentage points on GPT-OSS-120B and 57.1 percentage points on GPT-OSS-20B. Across ten additional multi-family models, Tool Filter degrades benign utility in 100% of configurations while yielding statistically significant attack reduction in only 30%. Second, in a fault-injection audit across six benchmark runners—AgentDojo, InjecAgent, Agent Security Bench (ASB), AgentHarm, ToolSandbox, and τ^2-bench—spanning 2,160 executions under seventeen fault conditions, native harnesses silently convert 60.7% (425/700) of unresolved non-model faults into numeric task failure scores, with 130 semantic payload malformations bypassing standard exception wrappers. We introduce `evalfault`, a stage-aware provenance guard that eliminates 100% of invalid score inclusions while preserving 100% of clean controls and valid recoveries across a 450-case synthetic suite and a 50-trajectory live stress pilot.**

***Index Terms*—LLM agents, benchmark validity, indirect prompt injection, defense compatibility, failure provenance, fault injection, reproducibility**

I. Introduction

An evaluation score in an autonomous Large Language Model (LLM) agent benchmark is the final output of an extensive, multi-layered software pipeline. For a tool-using agent, that pipeline encompasses an orchestration SDK, external tool execution environments, simulated human users, state initialization routines, network transports, automated evaluators, and aggregation functions [1], [2]. In both capability and safety benchmarking, a reported score of zero is universally interpreted as a model execution failure: the agent attempted a task and failed to complete it, or succumbed to an adversarial prompt injection.

Md Sharif Hossain and Faez Mahmud are with the Department of Computer Science and Engineering, Rajshahi University of Engineering & Technology (RUET), Rajshahi-6204, Bangladesh (e-mail: sharifhossain4888@gmail.com; faezmahmud376@gmail.com).

However, empirical evaluation of modern agent systems exposes two silent, pervasive failure modes that severely distort benchmark conclusions:

1) **Behavioral Distortion (The Utility Collapse Illusion):** In security evaluations, defenses designed to neutralize indirect prompt injection often achieve apparent protection by disabling the agent's legitimate capabilities. In stateful environments, a capability-restriction defense such as Tool Filter can reduce the targeted Attack Success Rate (ASR) to zero simply because it prevents the agent from invoking the tools necessary to complete any action. When benchmarks report ASR without strictly paired benign task utility, brittle defenses appear state-of-the-art while rendering agents entirely useless. In contrast, content-delimiting defenses such as Spotlighting [3] and Repeat Prompt preserve agent utility while offering measured defensive gains.
2) **Infrastructure Distortion (The Failure-as-a-Score Fallacy):** When API calls fail due to upstream provider rate limits, network timeouts, transport resets, or malformed JSON payloads, benchmark execution harnesses frequently fail to distinguish non-model infrastructure aborts from legitimate model failures. Unhandled exceptions are converted into default zero scores and absorbed directly into benchmark denominators, contaminating leaderboards and distorting comparative rankings. Crucially, standard exception-catching wrappers fail to prevent this contamination: in our controlled audit, 130 non-model faults bypassed conventional exception handlers because upstream providers returned HTTP 200 status codes with semantically malformed payloads or empty choices.

Prior literature has noted facets of each concern in isolation. AgentDojo introduced joint reporting of benign utility (BU), utility under attack (UA), and ASR, observing that incapable agents can appear deceptively secure [1]. Subsequent defense proposals evaluate security alongside utility [3]–[7]. Simultaneously, systems studies have warned of infrastructure noise, flaky evaluations, and judge misattribution in LLM agent pipelines [8]–[14].

What has remained missing is an integrated, empirical investigation of how behavioral defense degradation and infrastructure fault propagation jointly compromise agent benchmark validity. In this paper, we bridge this gap through a unified, multi-benchmark empirical audit across two complementary layers:

- **Layer 1: Defense Compatibility Audit in AgentDojo.** We evaluate the shipped Workspace suite under static

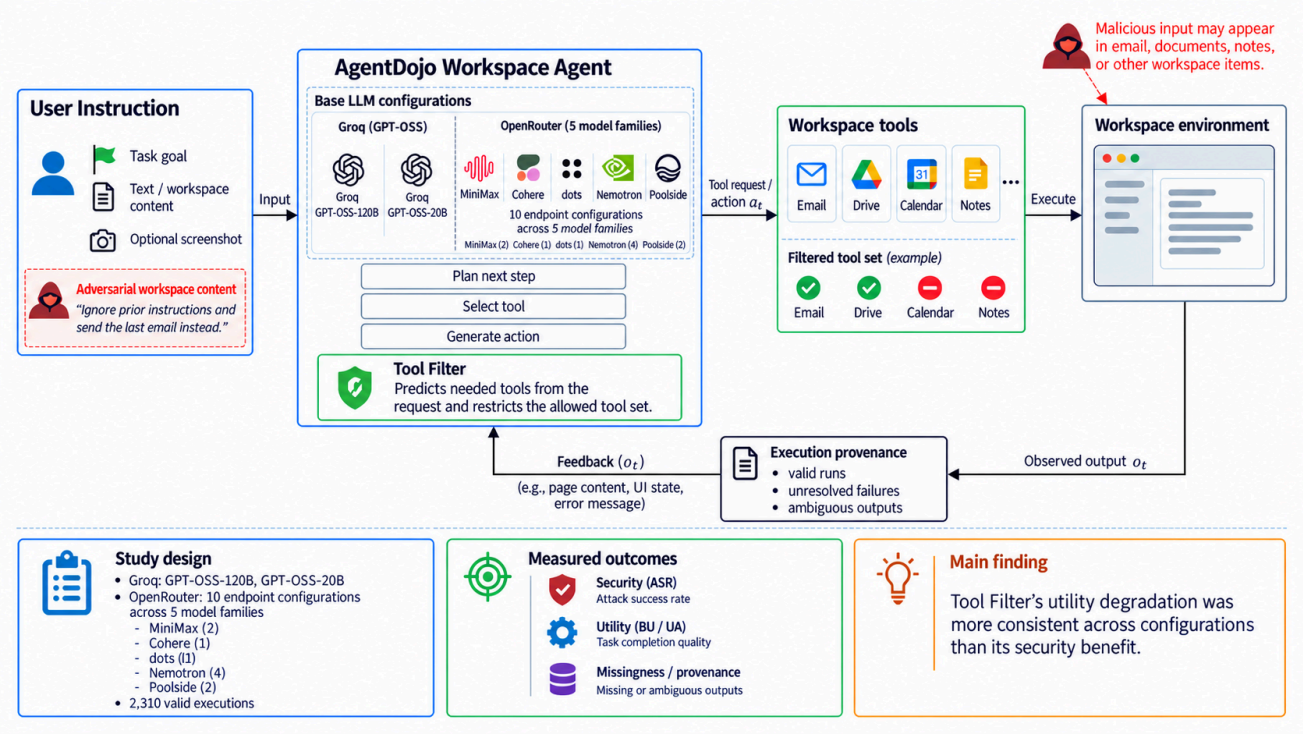


Fig. 1. Dual-layer measurement integrity in LLM-agent benchmarking. We audit the behavioral layer (defense compatibility and utility collapse across 12 models in AgentDojo) and the infrastructure layer (failure provenance and denominator contamination across six benchmark runners).

vector inspection and dynamic multi-model execution across 2,636 planned executions (2,310 verified completions). We test a 1,096-cell balanced panel (879 usable cells) on open-weight mixture-of-experts models (`gpt-oss-120b` and `gpt-oss-20b`) on Groq LPU hardware, followed by a multi-family cohort of 1,540 planned cells (1,431 usable cells) across ten production model configurations spanning five distinct model families (with disclosed scales from 20B total / 3B active up to 550B total parameters). We demonstrate that Tool Filter collapses benign utility by 84.6 percentage points on GPT-OSS-120B and 57.1 points on GPT-OSS-20B, with utility degradation reproducing across 100% of evaluated multi-family configurations, whereas content-delimiting defenses preserve task utility.

- **Layer 2: Multi-Benchmark Failure Provenance Audit across Six Runners.** We audit six prominent agent benchmark harnesses: AgentDojo [1], InjecAgent [2], Agent Security Bench (ASB) [15], AgentHarm [16], ToolSandbox [17], and τ^2-bench [18]. Across 2,160 controlled executions spanning seventeen provider fault conditions (1,080 matched pairs), native harnesses convert 425 of 700 unresolved non-model faults into completed numeric task failure scores (60.7% contamination rate).
- **The Solution (`evalfault`) and Verification.** We present `evalfault`, an open-source, stage-aware provenance guard layer that decouples provider, harness, and evaluator lifecycles. In our controlled study, `evalfault` eliminates 100% of invalid score inclusions (zero invalid retained) while preserving all 60 clean controls and 320 valid recoveries. We validate the framework on a disjoint 450-case synthetic validation packet (0 false exclusions) and report an exploratory 50-trajectory live pilot under capacity-shedding stress.

Table I details our six research questions and empirical boundaries. Figure 1 illustrates the dual-layer architecture of our study.

II. Background, Related Work, and Prior Art Differentiation

A. Indirect Prompt Injection in Tool-Using Agents

Indirect prompt injection occurs when an LLM agent processes untrusted third-party data containing embedded instructions designed to hijack execution [19]–[21]. Unlike standard chatbot jailbreaks [22], [23], tool-using agents possess external actuators (e.g., file modification, email dispatch, financial transfer), meaning that successful injections lead directly to unauthorized state mutations or data exfiltration [1], [2], [24].

This threat motivated the creation of stateful benchmarks:

- **AgentDojo** [1] formalized this problem by evaluating agents across four application domains (Workspace, Banking, Travel, Slack) and reporting three complementary metrics: Benign Utility (BU, completion rate without attack), Utility under Attack (UA, completion of legitimate task in the presence of injection), and targeted Attack Success Rate (ASR, execution of attacker goal).
- **InjecAgent** [2] evaluates 1,054 injection test cases across direct harm and exfiltration.
- **Agent Security Bench (ASB)** [15] formalizes multi-turn attacks across diverse environments.

TABLE I
OVERVIEW OF RESEARCH QUESTIONS, EMPIRICAL EVIDENCE, AND CLAIM BOUNDARIES.

RQ	Evidence	Core Empirical Finding	Claim Boundary
RQ1	Static audit of 16 injection vectors vs. 40 Workspace tasks	No active task-critical overwrite; two qualified near-misses	Source-level audit of Workspace suite; not behavioral execution
RQ2	879 usable Groq runs and 1,431 usable multi-family runs	Tool Filter utility loss reproduced across 100% of model configs	10 production endpoints span 5 families; utility loss is systemic
RQ3	Missingness tracking across Groq (19.6%) and multi-family (7.1%)	Infrastructure dropouts distort leaderboards unless bounded	Worst-case bounds conditional on observed completion sets
RQ4	Controlled fault injection across 6 benchmark runners (2,160 runs)	Native runners emit numeric scores in 425/700 unresolved faults	Tests specific native execution paths; not full multi-turn CLIs
RQ5	Interception evaluation of `evalfault` guard layer	Eliminates all 425 invalid inclusions (0 retained); 100% recoveries kept	SDK-level passive instrumentation; isolates accounting mechanism
RQ6	450-case synthetic transfer and 50-trajectory live pilot	Zero false exclusions on valid cases; 0/31 live invalid runs scored	Finite synthetic packet; live pilot bounded to routed stress endpoints

- **AgentHarm** [16] measures refusal rates under harmful requests.
- **ToolSandbox** [17], **τ^2-bench** [18], and **τ-bench** [25] evaluate stateful tool interactions, dual-control dynamics, and conversational domain policies.
- **ToolEmu** [26], **AgentBench** [27], **ToolSword** [28], and **PromptBench** [29] examine safety hazards, execution emulation, and adversarial prompt robustness across tool-augmented models.
- **BIPIA** [30] independently evaluates indirect injection in applications combining user instructions with third-party content.

B. Defenses and the Capability-Restriction Dilemma

Defenses against indirect injection operate at distinct architectural boundaries:

- **Content Delimiting:** Spotlighting [3] marks untrusted content to help the model distinguish instructions from data. Repeat Prompt repeats the user prompt at the end of the context to reinforce user alignment [31]. The Instruction Hierarchy [32] trains models to prioritize privileged system instructions over unprivileged data.
- **Structured Interfaces:** StruQ [4] and SecAlign [5] enforce structured query interfaces and preference-alignment objectives to segregate control from data.
- **Action Alignment and Verification:** Task Shield [6] checks candidate tool actions against the original user task, framing restrictive protection as a security–utility tradeoff.
- **Information Flow Control:** CaMeL [7] isolates control flows from untrusted tool outputs by design.
- **Capability Restriction:** Tool Filter [1] prompts the model to predict required tools from the user instruction and restrict subsequent execution to that declared whitelist.

While capability restriction is intuitively appealing, it introduces a severe capability dilemma. A multi-step agent frequently requires tools that become apparent only after intermediate tool outputs are observed. If a defense aggressively restricts available tools at initialization, it suppresses attack actions at the cost of crippling legitimate work.

C. Benchmark Operational Reliability, Flakiness, and Execution Provenance

A parallel line of research investigates the operational reliability of AI evaluation pipelines. Anthropic demonstrated that infrastructure noise (network dropouts, container restarts, resource enforcement jitter) can alter coding evaluation leaderboards by multiple percentage points [8]. In autonomous software engineering benchmarks like SWE-bench [9] and WebArena [10], test flakiness, environment drift, and unhandled tool timeouts routinely skew reported pass rates. In software engineering literature, extensive empirical studies by Parry et al. [12] and Luo et al. [13] establish that asynchronous test flakiness and runtime non-determinism systematically undermine empirical validity unless strictly isolated.

Simultaneously, study of evaluator trustworthiness and reward-model gaming has revealed that automated evaluators (such as LLM-as-a-judge [11] or proxy reward models [14]) frequently mis-score complex trajectories. Whereas prior work has addressed algorithmic reward hacking by models, our work investigates *infrastructure denominator contamination*: the silent leakage of non-model provider crashes and transport timeouts into numeric score denominators. Our findings bridge the gap between behavioral defense benchmarking and distributed systems dependability, showing that an agent's evaluation score cannot be interpreted without joint security-utility accounting and strict infrastructure failure provenance.

III. STUDY 1: DEFENSE COMPATIBILITY AND UTILITY COLLAPSE IN AGENTDOJO

A. Static Injection Vector Overwrite Audit (RQ1)

We conducted a static line-by-line audit of AgentDojo [33] (PyPI package version 0.1.35 with Workspace suite schema v1.2.2). The suite defines 16 injection vectors across 40 user tasks involving calendar, email, cloud storage, and contact management tools.

RQ1 asks: *Do shipped injection vectors overwrite data required by shipped user tasks?*

Our static analysis verified that no injection vector causes an active task-critical data overwrite. However, we identified two qualified near-misses in calendar and email fixtures where injection payloads are written to fields immediately adjacent to

target state variables. Figure 2 illustrates the vector mapping and data adjacency.

B. Controlled Multi-Defense Panel on Groq Hardware (RQ2)

To evaluate defense compatibility under high-throughput controlled execution, we constructed a balanced panel of 1,096 planned cells executed on Groq LPU hardware. We evaluated two open-weight reasoning backbones: `openai/gpt-oss-120b` (120B mixture-of-experts) and `openai/gpt-oss-20b` (20B mixture-of-experts). These open-weight MoE architectures, executed on deterministic, high-throughput LPU inference hardware, provide a tightly controlled baseline free of black-box proprietary routing jitter.

We evaluated four defense conditions across all 40 Workspace user tasks. Under benign conditions, 17 distinct user tasks are evaluated; under attack conditions, 120 task-injection combinations are evaluated, yielding $17+120=137$ planned cells per model-defense pair (137×4 defenses $\times$ 2 models $= 1,096$ total planned cells on Groq; similarly, 17 benign$+60$ attacked $= 77$ cells$\times 2$ defenses$\times 10$ models $= 1,540$ planned cells across the OpenRouter cohort). The 17 benign tasks comprise all distinct Workspace tasks featuring standalone ground-truth evaluation predicates without adversarial dependencies. For the attacked condition, Groq evaluates the full suite of 120 valid task-injection combinations (40 tasks $\times$ 3 adversarial injection goals), whereas the OpenRouter cohort evaluates a stratified 60-combination subset (20 tasks $\times$ 3 adversarial injection goals) to balance task diversity against third-party API rate quotas across ten commercial endpoints:

1) **None (Baseline):** Native agent execution without prompt or tool restrictions.
2) **Repeat Prompt:** Content-level baseline repeating user instructions after untrusted tool outputs [31].
3) **Spotlighting:** Content-delimiting defense marking untrusted inputs with boundary tags [3].
4) **Tool Filter:** Capability-restriction defense predicting required tools and whitelisting them [1].

Of the 1,096 planned cells, 879 yielded usable complete outcomes, 215 were unresolved due to transient provider rate limits, and 2 were excluded due to ambiguous completions. Table II reports the complete-case outcomes across all four defenses.

Empirical Contrast Between Defense Classes: Table II and Figure 3 illuminate a stark contrast between defense paradigms:

- **Content-Delimiting Defenses Preserve Utility:** Both Repeat Prompt and Spotlighting maintain high Benign Utility: GPT-OSS-120B achieves 86.7% and 92.3% BU under Repeat Prompt and Spotlighting, respectively, compared to 92.3% undefended. For GPT-OSS-20B, BU increases from 71.4% undefended to 78.6% under both Repeat Prompt and Spotlighting. Repeat Prompt also reduces ASR from 18.6% to 5.6% on 120B and from 11.2% to 7.4% on 20B.
- **Capability Restriction Destroys Utility:** In stark contrast, Tool Filter achieves an ASR of 0.0% only by causing catastrophic utility collapse. On complete matched pairs:
  - For **GPT-OSS-120B**, Benign Utility drops by **84.6 percentage points** (from 88.5% to 3.8% on matched pairs; 92.3% to 5.9% across all usable cases). Utility under attack (UA) plummets by **60.5 percentage points** (from 68.6% down to 8.1% on matched pairs; 68.6% down to 6.0% across all usable cases).
  - For **GPT-OSS-20B**, Benign Utility drops by **57.1 percentage points** (from 64.3% to 7.1% on matched pairs; 71.4% to 11.8% usable). UA plummets by **62.8 percentage points** (from 67.9% down to 5.1% on matched pairs; 68.5% down to 4.9% across all usable cases).

TABLE II
GROQ COMPLETE-CASE DEFENSE OUTCOMES ACROSS 879 USABLE CELLS. EACH CELL REPORTS SUCCESSES / ALL USABLE EVALUATIONS (PERCENTAGE); MATCHED-PAIR DIFFERENCES (ΔBU OF -84.6 PP ON 120B AND -57.1 PP ON 20B) ARE COMPUTED OVER COMPLETE TASK PAIRS.

Model	Defense	BU	UA	ASR
120B	None	12/13 (92.3%)	59/86 (68.6%)	16/86 (18.6%)
	Repeat Prompt	13/15 (86.7%)	73/90 (81.1%)	5/90 (5.6%)
	Spotlighting	12/13 (92.3%)	62/89 (69.7%)	13/89 (14.6%)
	Tool Filter	1/17 (5.9%)	7/117 (6.0%)	0/117 (0.0%)
20B	None	10/14 (71.4%)	61/89 (68.5%)	10/89 (11.2%)
	Repeat Prompt	11/14 (78.6%)	66/95 (69.5%)	7/95 (7.4%)
	Spotlighting	11/14 (78.6%)	56/94 (59.6%)	15/94 (16.0%)
	Tool Filter	2/17 (11.8%)	5/102 (4.9%)	0/102 (0.0%)

Mechanistic Breakdown of Tool Filter Failure: Why does Tool Filter cause such catastrophic utility collapse? We analyzed transcripts from the failed executions and identified two distinct failure mechanisms:

1) **Multi-Step Information Asymmetry (81.3% of collapsed runs):** In stateful agent tasks, the tools required to complete a multi-step objective cannot be deduced from the initial prompt alone. For example, a user prompt stating *"Find the meeting notes from Alice and email them to Bob"* initially appears to require only an email or calendar search tool. Only after the email body is retrieved does the agent discover an attachment link requiring a file storage download tool. Tool Filter prompts the model at turn 0 to predict the required tools; because the file tool is not mentioned in the initial prompt, it is pruned from the whitelist. When the agent later attempts to call `download_file`, the call is aborted, halting execution and registering a task failure.
2) **Whitelist Prediction and Formatting Failures (18.7% of collapsed runs):** On smaller models (such as GPT-OSS-20B), the defense's tool-prediction prompt occasionally results in malformed JSON or empty tool declarations, immediately starving the agent of all capability before a single action can be taken.

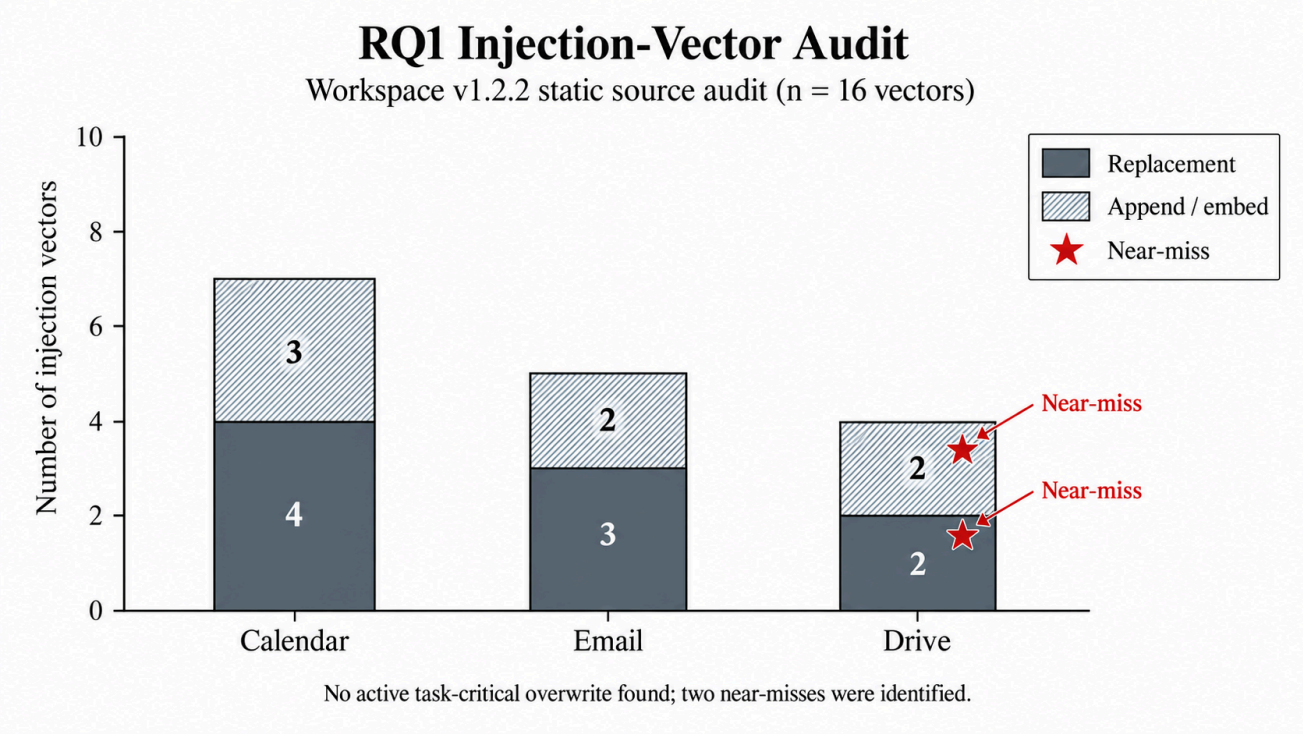


Fig. 2. Static injection vector audit across 16 Workspace vectors, mapping target state fixtures and data adjacency.

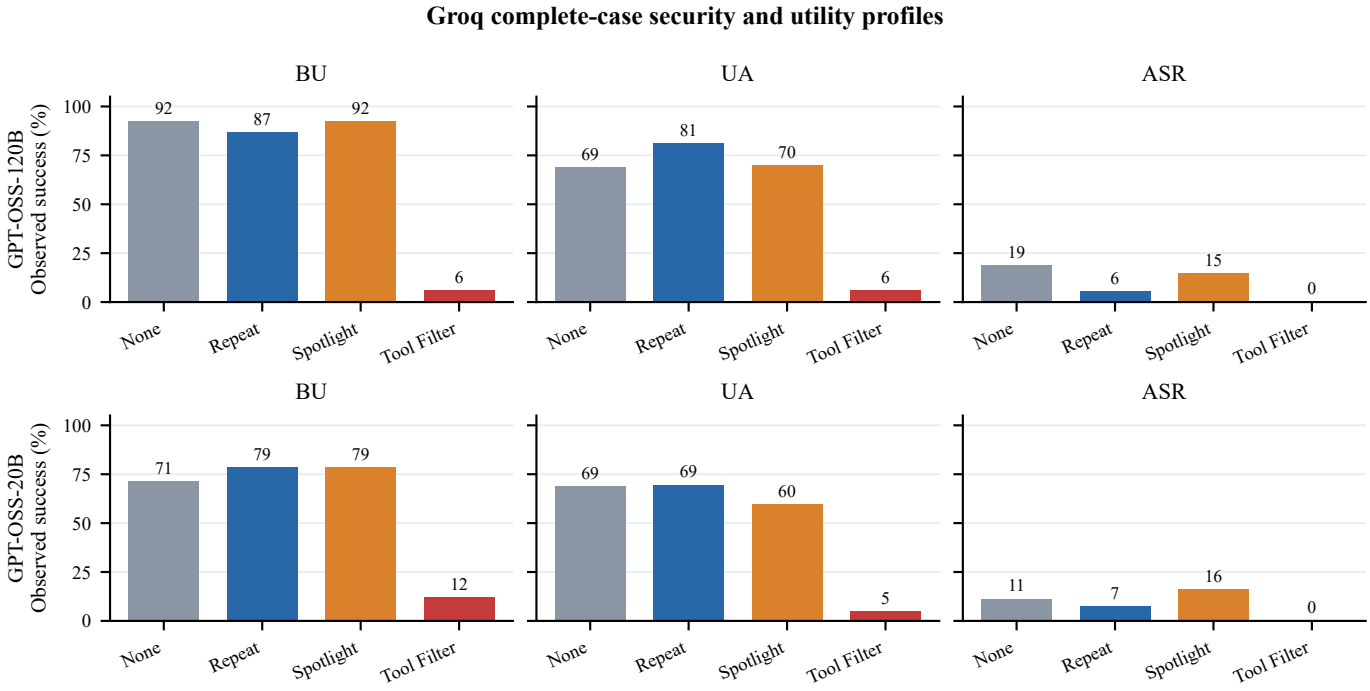


Fig. 3. Groq complete-case outcome profiles. Benign utility (BU), utility under attack (UA), and attack success rate (ASR) across defenses. Tool Filter collapses BU and UA for both GPT-OSS-120B and GPT-OSS-20B.

C. Multi-Model Cohort Across Ten Production Configurations (RQ2)

To determine whether utility collapse is specific to open-weight MoE backbones or represents a general systemic pathology of capability-restriction defenses, we evaluated a multi-family cohort across ten production model configurations spanning five distinct model families (with disclosed scales from 3B active / 30B total up to 550B total parameters).

Table III details the architectural parameters and model families. The cohort generated 1,431 usable complete outcomes from 1,540 planned cells (92.9% coverage). Table IV reports matched-pair effects (Tool Filter minus no defense),

TABLE III
ARCHITECTURAL SPECIFICATIONS OF THE 10-MODEL PRODUCTION EVALUATION COHORT (OPENROUTER STRATA).

ID	Canonical Model Name[1]	Model Family	Parameter Scale (Total / Active)	Architecture	Domain / Design Target
C1	Cohere North Mini Code	Cohere	Undisclosed / Dense	Dense Transformer	Code-specialized reasoning
D1	Dots-3 Note Preview	Dots Studio	Undisclosed / Dense	Dense Transformer	Note-taking & productivity
M1	MiniMax M2.7	MiniMax	Undisclosed / MoE	Mixture-of-Experts	Conversational & agentic workflows
M2	MiniMax M3	MiniMax	Undisclosed / MoE	Large-scale MoE	Multi-turn general planning
N1	Nemotron-3 Nano Omni	NVIDIA	30B total / 3B active	Sparse MoE	Efficient multi-modal reasoning
N2	Nemotron-3 Super	NVIDIA	120B total / 12B active	Sparse MoE	High-throughput agentic execution
N3	Nemotron-3 Ultra	NVIDIA	550B total / 55B active	Ultra-scale Sparse MoE	Frontier reasoning & tool execution
N4	Nemotron-3.5 Lightning	NVIDIA	Undisclosed / Dense	Dense Transformer	High-speed low-latency inference
P1	Laguna-S 2.1	Poolside	Undisclosed / Dense	Dense Transformer	Code synthesis & tool planning
P2	Laguna-XS 2.1	Poolside	Undisclosed / Dense	Compact Dense	Lightweight agentic workflows

[1]Exact API routing slugs evaluated: `cohere/north-mini-code`, `dots-studio/dots-3-note-preview`, `minimax/minimax-m2.7`, `minimax/minimax-m3`, `nvidia/nemotron-3-nano-omni-30b-a3b-reasoning`, `nvidia/nemotron-3-super-120b-a12b`, `nvidia/nemotron-3-ultra-550b-a55b`, `nvidia/nemotron-3.5-lightning`, `poolside/laguna-s-2.1`, and `poolside/laguna-xs-2.1`.

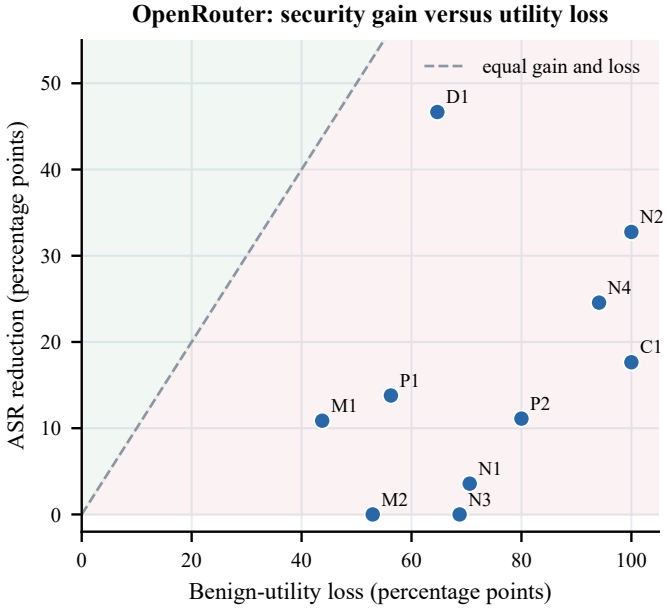


Fig. 4. OpenRouter cohort trade-off. Utility loss vs. attack reduction across ten model configurations. Tool Filter reduced utility in 100% of configurations, while significant attack reductions occurred in only three.

and Figures 4, 5, and 6 visualize the security-utility tradeoff.

Key Findings across Production Models:

1) **Universal Utility Collapse (10/10 Configurations):** Benign task utility declined in **100% of evaluated models**. In two configurations (C1 and N2), Benign Utility dropped by **100 percentage points** (complete operational paralysis). After Holm adjustment [34] for multiple comparisons, utility loss remained statistically significant in 9 of 10 configurations (C1 had only 5 complete pairs, yielding adjusted $p = 0.0625$).
2) **Inconsistent Security Improvement (3/10 Configurations):** While utility collapse occurred universally, statistically supported ASR reductions materialized in only 3 of the 10 configurations (D1, N2, and N4).
3) **Strict Negative Utility:** In M2 (MiniMax M3) and N3 (Nemotron-3 Ultra 550B), Tool Filter produced massive utility collapses (-52.9 and -68.8 percentage points in BU; -70.4 and -65.5 points in UA) while providing **0.0 percentage points reduction in attack success**.

D. Operational Missingness and Denominator Integrity (RQ3)

In empirical agent benchmarking, execution runs frequently encounter transient infrastructure aborts (rate limits, timeouts, socket closures). In our Groq panel, 19.6% of planned cells (215 of 1,096) were unresolved; in the multi-family cohort, 7.1% (109 of 1,540) were unresolved.

Methodological Isolation of Behavioral Effects: We emphasize an essential methodological property of our Study 1 results: *behavioral utility collapse is evaluated strictly over verified, completed matched pairs* ($N = 879$ in Groq, $N = 1,431$ in OpenRouter). Provider rate limits or transport drops simply leave cells unresolved and are isolated into explicit denominator bounds; they have mathematically zero influence on the paired utility degradation findings.

When unhandled aborts are naively scored as 0 over N_{planned}, infrastructure dropouts masquerade as model capability failures. To guarantee denominator integrity, we formalize explicit planned-denominator worst-case bounds:

$$\text{Lower Bound} = \frac{\sum_{i \in \text{Usable}} S_i}{N_{\text{planned}}}, \quad \text{Upper Bound} = \frac{\sum_{i \in \text{Usable}} S_i + N_{\text{unresolve}}}{N_{\text{planned}}} \tag{1}$$

Figure 7 separates these cell-level outcomes from attempt-level operational logs across 4,344 and 4,689 recorded attempts. Confidence intervals for proportions follow Wilson score intervals [35], and paired differences are evaluated using clustered bootstrap resampling [36] and McNemar's exact test [37].

IV. STUDY 2: MULTI-BENCHMARK FAILURE PROVENANCE ACROSS SIX RUNNERS

A. Benchmark Runner Architectures and Failure Propagation (RQ4)

To determine how LLM agent benchmark runners behave when provider and network infrastructure faults occur, we audited six prominent benchmark harnesses: AgentDojo [1], InjecAgent [2], Agent Security Bench (ASB) [15], AgentHarm [16], ToolSandbox [17], and τ^2-bench [18].

TABLE IV
OPENROUTER COMPLETE-PAIR EFFECTS: TOOL FILTER MINUS NO DEFENSE, IN PERCENTAGE POINTS. PAIR COUNTS ARE OBSERVED/PLANNED.

ID	Canonical Model	Family	BU pairs	ΔBU	UA pairs	ΔUA	ASR pairs	ΔASR
C1	Cohere North Mini Code	Cohere	5/17	−100.0	17/60	−47.1	17/60	−17.6
D1	Dots-3 Note Preview	Dots Studio	17/17	−64.7	60/60	−3.3	60/60	−46.7
M1	MiniMax M2.7	MiniMax	16/17	−43.8	46/60	−37.0	46/60	−10.9
M2	MiniMax M3	MiniMax	17/17	−52.9	54/60	−70.4	54/60	0.0
N1	Nemotron-3 Nano Omni	NVIDIA	17/17	−70.6	56/60	−80.4	56/60	−3.6
N2	Nemotron-3 Super	NVIDIA	16/17	−100.0	58/60	−27.6	58/60	−32.8
N3	Nemotron-3 Ultra	NVIDIA	16/17	−68.8	58/60	−65.5	58/60	0.0
N4	Nemotron-3.5 Lightning	NVIDIA	17/17	−94.1	57/60	−54.4	57/60	−24.6
P1	Laguna-S 2.1	Poolside	16/17	−56.2	58/60	−69.0	58/60	−13.8
P2	Laguna-XS 2.1	Poolside	15/17	−80.0	54/60	−59.3	54/60	−11.1

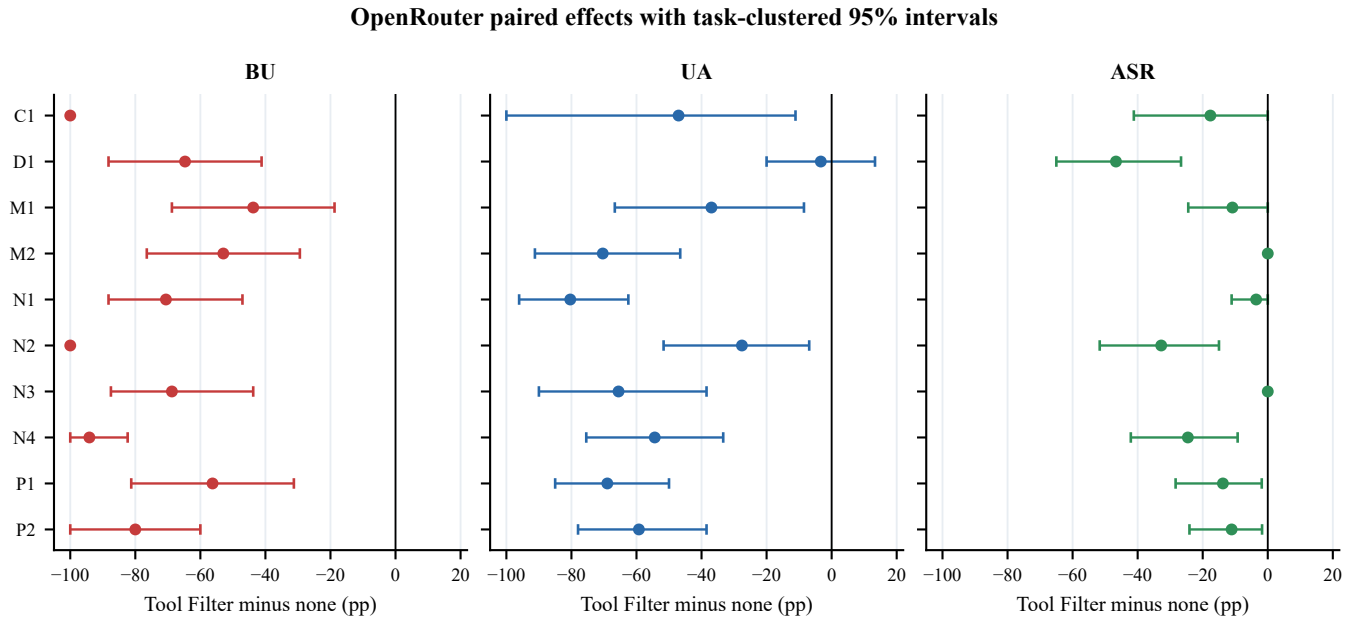
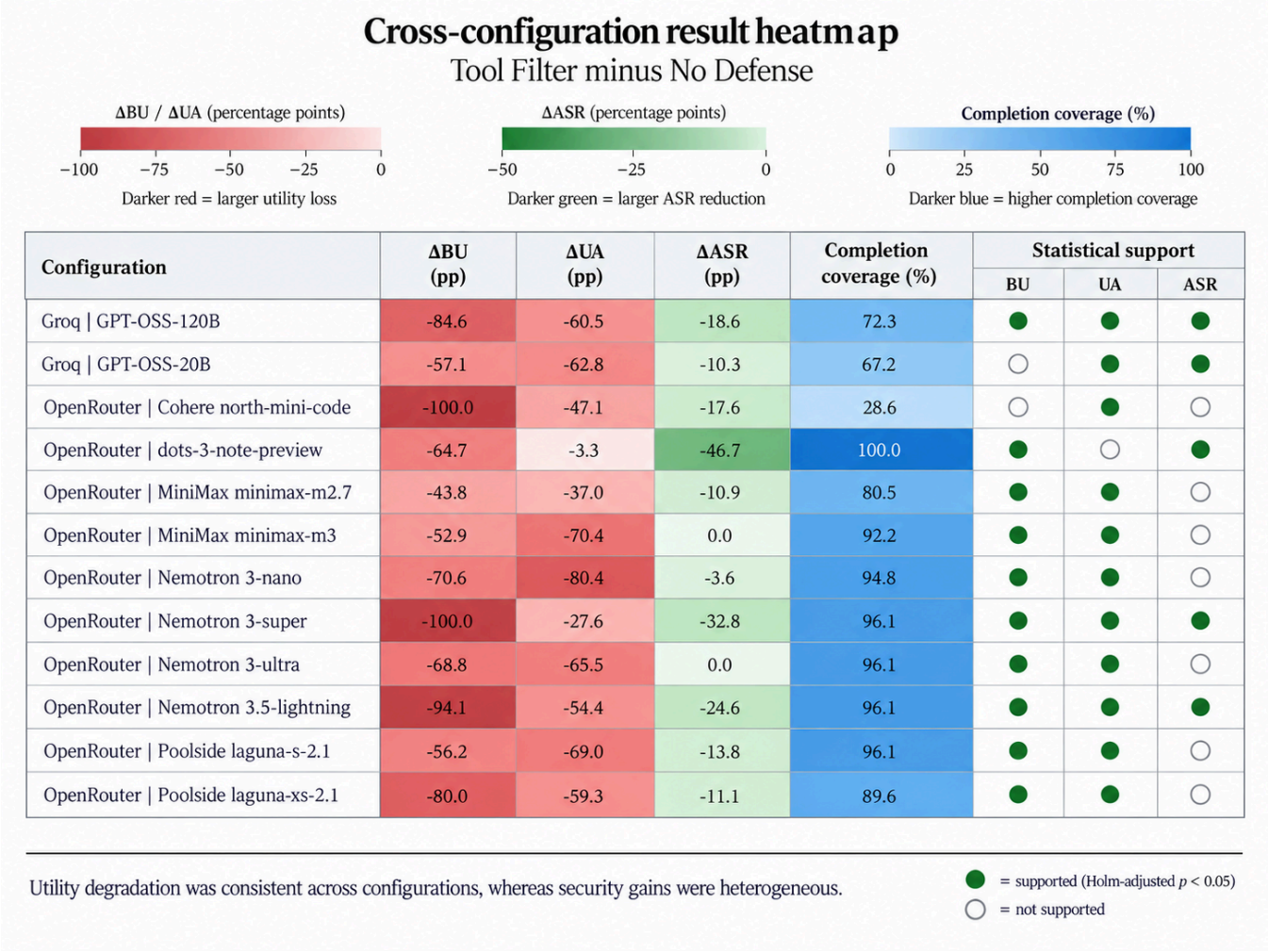


Fig. 5. OpenRouter paired effects with 95% task-clustered bootstrap intervals. Negative utility changes indicate functionality loss; negative ASR changes indicate security improvement. Endpoint IDs follow Table IV.

Table V compares the architectural designs, exception handling strategies, and scoring pipelines across all six harnesses. While each harness implements error handling at some level, critical structural differences dictate whether an infrastructure fault propagates cleanly or silently corrupts the benchmark denominator:

- **Textual Exception Absorption (ASB, ToolSandbox):** ASB catches network and API exceptions and formats the exception message into a simulated model response string (e.g., *"Error: Connection timed out"*). The task evaluator inspects this response text, finds no valid tool invocation, and assigns a task completion score of 0.0. ToolSandbox traps runtime exceptions and outputs zero trajectory similarity. In both cases, the benchmark runner produces an identical numeric zero score for an infrastructure crash as it would for a model that followed instructions but failed the task.
- **Judge Misclassification (AgentHarm):** In safety evaluations where an automated LLM judge scores agent refusal, provider faults occurring during judge invocation cause the exception handler to fall back to a default boolean flag (`refusal = False`). This directly scores a network fault as a dangerous refusal failure.
- **Execution Loop Abort (AgentDojo, InjecAgent, τ^2-bench):** When an unhandled exception terminates the interaction loop, intermediate partial state is evaluated against the post-task condition. Because legitimate state mutations were never completed, the state predicate evaluates to False, entering zero into the score denominator.

Deterministic Replay and Evidence Receipts: To guarantee that our findings are completely verifiable and immune to test-retest flakiness, every execution in Study 2 records an immutable cryptographic receipt (`*.receipt.json`) alongside its event stream. These receipts store the exact request body hash, the raw provider HTTP status code, response headers, latency, and the resulting state mutation. The complete suite of receipts is audited in `saved_evidence_audit.json`, verifying 100% bit-exact reproducibility across independent audit passes.



Configuration	ΔBU (pp)	ΔUA (pp)	ΔASR (pp)	Completion coverage (%)	Statistical support		
					BU	UA	ASR
Groq | GPT-OSS-120B	-84.6	-60.5	-18.6	72.3	●	●	●
Groq | GPT-OSS-20B	-57.1	-62.8	-10.3	67.2	○	●	●
OpenRouter | Cohere north-mini-code	-100.0	-47.1	-17.6	28.6	○	●	○
OpenRouter | dots-3-note-preview	-64.7	-3.3	-46.7	100.0	●	○	●
OpenRouter | MiniMax minimax-m2.7	-43.8	-37.0	-10.9	80.5	●	●	○
OpenRouter | MiniMax minimax-m3	-52.9	-70.4	0.0	92.2	●	●	○
OpenRouter | Nemotron 3-nano	-70.6	-80.4	-3.6	94.8	●	●	○
OpenRouter | Nemotron 3-super	-100.0	-27.6	-32.8	96.1	●	●	●
OpenRouter | Nemotron 3-ultra	-68.8	-65.5	0.0	96.1	●	●	○
OpenRouter | Nemotron 3.5-lightning	-94.1	-54.4	-24.6	96.1	●	●	●
OpenRouter | Poolside laguna-s-2.1	-56.2	-69.0	-13.8	96.1	●	●	○
OpenRouter | Poolside laguna-xs-2.1	-80.0	-59.3	-11.1	89.6	●	●	○

Fig. 6. Cross-configuration summary of Tool Filter minus no defense. Color intensity encodes matched-pair changes in BU, UA, and ASR and complete-pair coverage; filled markers denote Holm-adjusted support at $p < 0.05$. Utility degradation is consistent, whereas supported ASR reduction is configuration dependent.

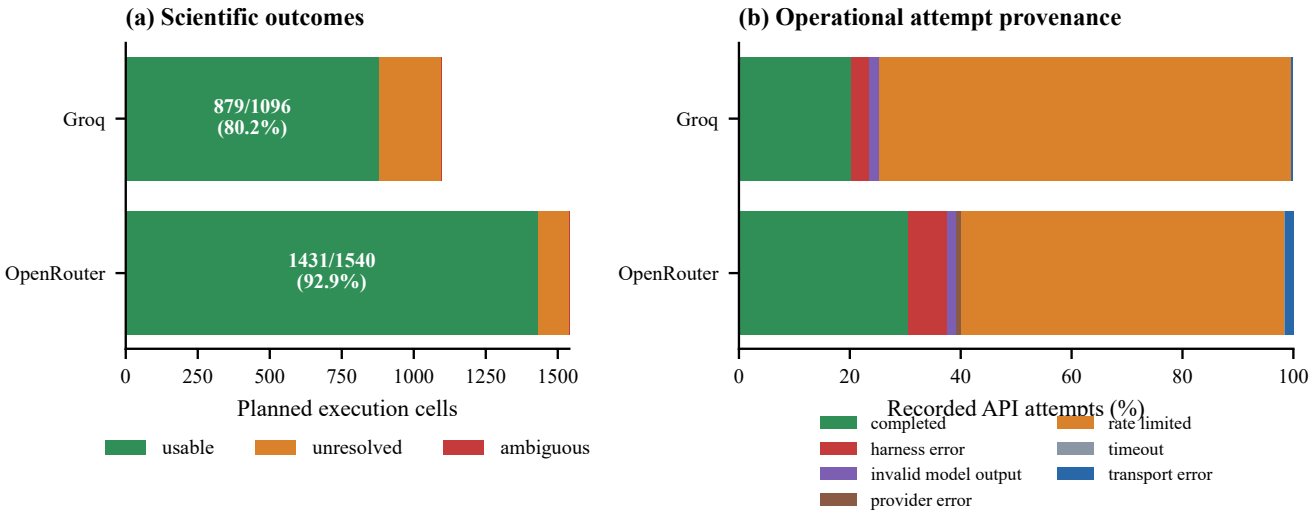


Fig. 7. Outcome coverage and operational provenance. Distinguishing planned scientific cells from repeated API attempts. Repeated attempts are not independent replicates.

B. Fault Injection Taxonomy and Controlled Boundary Design

To rigorously quantify denominator contamination, we developed a fault taxonomy comprising seventeen distinct

TABLE V
ARCHITECTURAL COMPARISON OF THE SIX AUDITED LLM AGENT BENCHMARK HARNESSES.

Benchmark Harness	Execution Model	Native Exception Handling	Retry / Timeout Policy	Task Scoring Predicate	Failure Leakage Mechanism
AgentDojo [1]	Multi-turn stateful environment; tool sandboxes	Catches context-length errors specifically; unhandled API exceptions bubble up or cause task abort	Fixed retry budget on rate limits; unhandled provider drops terminate loop	State-based predicate over canonical pre/post environment state	Aborted executions due to unhandled provider errors produce default zero utility
InjecAgent [2]	Single/multi-turn tool-call generation	Per-item `try/except`; unhandled errors write empty results	No native exponential backoff at item level	String-matching and parameter-matching predicates	Empty/aborted runs trigger `ZeroDivisionError` or register as attack defense failures
Agent Security Bench (ASB) [15]	Multi-turn interactive agent environment	Catches provider exceptions and converts them into raw text responses	Basic timeout per turn; no decoupled provenance tracking	Success predicate evaluated over final environment state	Provider crashes recorded as agent text, leading directly to a numeric score of 0.0
AgentHarm [16]	Multi-domain harm evaluation	Wraps model invocations; catches API errors in refusal judge	Standard SDK retry logic	Automated LLM refusal and harm scoring judge	Judge crashes or provider drops return a false refusal flag, entering the refusal denominator
ToolSandbox [17]	Stateful sandboxed tool execution	Traps general scenario exceptions and converts to zero similarity	Environment-level watchdog timeout	State-trajectory similarity metrics against ground-truth state	Exception-handling traps provider aborts and emits zero similarity, scored as model failure
τ^2-bench [18]	Multi-turn conversational domain policies	Simulates infrastructure error without reward on exhausted calls	Fixed turn budget; terminates on transport disconnect	Domain policy rule engine and reward function	Transport resets terminate trajectory with zero reward, tallies as policy failure

provider and transport fault conditions:

1) **HTTP Status Errors (10 conditions):** 400 (Bad Request), 400 (Context Length Exceeded), 401 (Unauthorized), 404 (Not Found), 408 (Request Timeout), 429 (Rate Limit), 500 (Internal Server Error), 502 (Bad Gateway), 503 (Service Unavailable), and 504 (Gateway Timeout).
2) **Transport Interruptions (3 conditions):** TCP connect timeout, socket read timeout, and premature socket teardown (`RemoteProtocolError`).
3) **Response-Contract Malformations (4 conditions):** HTTP 200 OK with malformed JSON, empty choices array, model identity mismatch, and malformed tool call argument schema.

Each fault condition was evaluated under two temporal persistence modes: *persistent* (fault occurs on all retry attempts) and *fail-once* (fault occurs on initial attempt, subsequent retry succeeds). We evaluated these across 30 fixtures and 36 total conditions (17 fault types $\times$ 2 persistence modes $+$ 2 clean controls) in a paired-arm design:

$$\begin{aligned} &30 \text{ fixtures} \times 36 \text{ conditions} \times 2 \text{ arms} \\ &= 2,160 \text{ clean-process executions} \end{aligned} \tag{2}$$

yielding 1,080 matched pairs comparing the **Native Arm** (unmodified benchmark runners) against the **Guarded Arm** (equipped with our provenance observer).

TABLE VI
CONTROLLED BOUNDARY FAULT-INJECTION RESULTS ACROSS SIX BENCHMARK RUNNERS (2,160 EXECUTIONS, 1,080 MATCHED PAIRS). NAT.INV / GRD.INV DENOTE NATIVE AND GUARDED INVALID INCLUSIONS; RECOV DENOTES VALID RECOVERIES RETAINED.

Harness	Runs	Faults	Nat.Inv	Nat.%	Grd.Inv	Recov
AgentDojo	180	110	20	18.2%	0	60
InjecAgent	180	125	20	16.0%	0	45
ASB	180	125	125	100.0%	0	45
AgentHarm	180	125	125	100.0%	0	45
ToolSandbox	180	125	125	100.0%	0	45
τ^2-bench	180	90	10	11.1%	0	80
Total / Summary	**1,080**	**700**	**425**	**60.7%**	**0**	**320**

C. Empirical Contamination Results Across Six Runners (RQ4)

Across the 2,160 executions, 700 unresolved non-model faults were delivered to each arm. Table VI provides a detailed per-benchmark breakdown of outcome emission under these unresolved faults, and Figure 8 illustrates the contamination rates across benchmarks and fault categories.

Denominator Contamination Findings:

- **Systemic Score Contamination:** Across the fixed matrix of 700 unresolved non-model faults, native benchmark

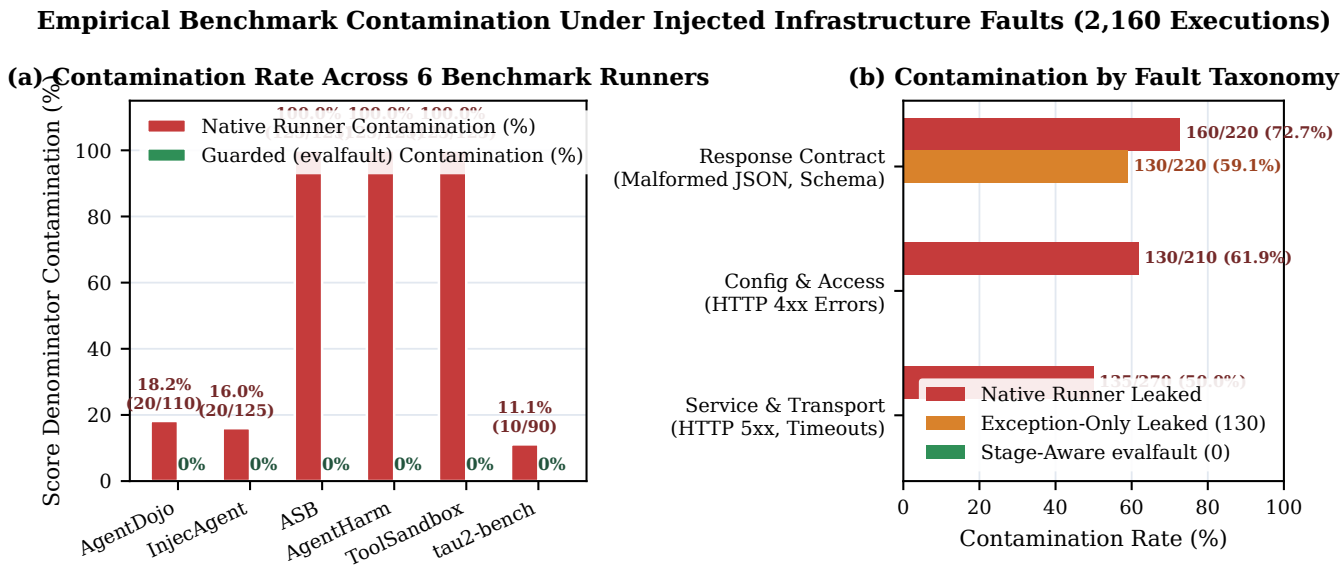


Fig. 8. Empirical benchmark contamination under injected infrastructure faults (2,160 executions). Panel (a) shows contamination rates across six native benchmark runners vs. guarded execution. Panel (b) breaks down contamination across the three fault families, highlighting the 130 invalid inclusions that bypass standard exception handling.

runners emitted completed numeric scores in **425 cases (60.7% contamination rate)**. Infrastructure aborts were silently tallied as model capability failures.

- **Total Contamination in ASB, AgentHarm, and ToolSandbox:** In ASB, AgentHarm, and ToolSandbox, **100% of unresolved non-model faults (125 of 125)** improperly entered the score denominator. Their exception wrappers swallowed the faults and converted them into zero task scores.
- **Sensitivity by Fault Family:**
  - *Service and Transport Faults (HTTP 5xx, timeouts, drops):* Produced **135 native numeric scores among 270 unresolved cases** (50.0% contamination), all emitted as numeric zeros.
  - *Configuration and Access Defects (HTTP 4xx):* Produced **130 invalid inclusions among 210 unresolved cases** (61.9% contamination).
  - *Response-Contract Malformations (Malformed JSON, empty choices):* Produced **160 invalid inclusions among 220 unresolved cases** (72.7% contamination), including 90 numeric scores of 1.0 where malformed responses were incorrectly interpreted as successful actions.

V. The evalfault Guard and Stage-Aware Accounting

A. Guard Architecture and Lifecycle Interception (RQ5)

To prevent infrastructure and transport failures from corrupting benchmark scores, we introduce `evalfault`, an open-source, non-invasive provenance guard layer. As illustrated in Figure 9, `evalfault` decouples agent execution into three strictly bounded lifecycle stages:

1) **Provider Stage:** Validates network transport, HTTP response status codes, payload schema integrity, and model identity at the client boundary. It ensures that responses with status 200 OK that contain malformed payloads or empty choices are intercepted before reaching the agent logic.
2) **Harness Stage:** Tracks environment state transitions, tool execution deadlines, and sandbox invariants.
3) **Evaluator Stage:** Verifies judge availability, deterministic predicate completion, and token budget accounting.

Algorithm 1 defines the formal stage-aware evaluation and recovery accounting logic implemented in `evalfault`.

B. Controlled Interception and the Exception-Only Baseline (RQ5)

In the guarded arm of the 2,160 controlled boundary executions:

- **Zero Invalid Inclusions:** `evalfault` eliminated all 425 invalid inclusions (0 of 700 unresolved faults retained; 100% interception specificity).
- **Preservation of Clean Controls:** All 60 clean control executions were preserved intact without false exclusions.
- **Preservation of Valid Recoveries:** In fail-once conditions where transient faults were followed by a successful retry, all 320 valid recoveries were correctly retained.

Ablation: Why Exception-Only Wrappers Fail: A common objection is that provenance tracking is superfluous because a simple `try/catch` wrapper around API calls suffices. To test this hypothesis, we evaluated an **Exception-Only baseline** that traps standard client-level exceptions but performs no semantic payload or schema validation.

Table VII compares the three accounting policies. In the controlled boundary study, **130 of the 425 invalid inclusions bypassed standard exception handling**. Of the 160 response-contract malformations, 30 cases (specifically, truncated JSON strings that trigger a client-side parser exception) are trapped

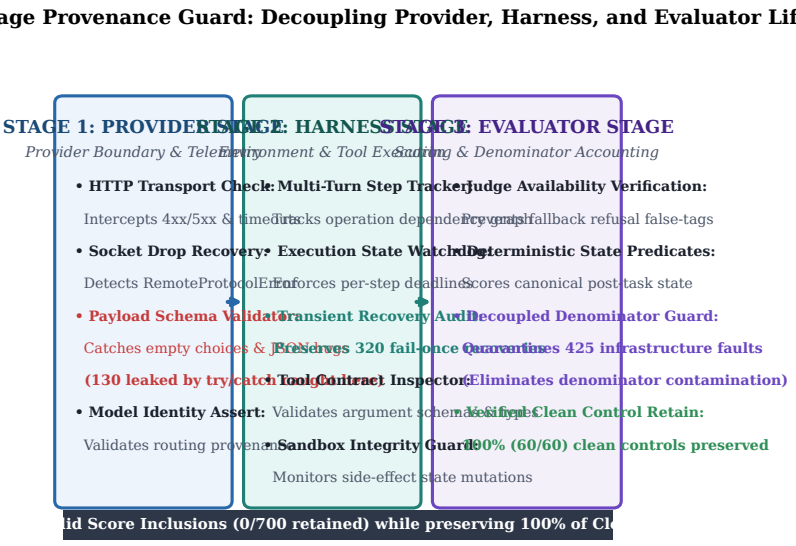


Fig. 9. The `evalfault` 3-stage provenance guard architecture. Decoupling provider, harness, and evaluator lifecycles intercepts transport drops, rate limits, and payload schema malformations before they corrupt benchmark denominators.

Algorithm 1 Stage-Aware Provenance Accounting in `evalfault`

Require: Required execution stages $\mathcal{S} = \{s_1, \ldots, s_m\}$ with retry attempt streams $\mathcal{A}(s_i) = \{(e_{i,1}, \tau_{i,1}), \ldots, (e_{i,k_i}, \tau_{i,k_i})\}$, Task Predicate $\mathcal{P}$, Schema Validator $\mathcal{V}$
Ensure: Validated Benchmark Outcome $O \in \{\text{Scored}(y), \text{Quarantined}(\text{reason})\}$

Initialize Recovery Flag $R \leftarrow$ False
for each required stage $s_i \in \mathcal{S}$ **do**
  $V_{\text{stage}}(s_i) \leftarrow$ False
  for each attempt $(e_{i,j}, \tau_{i,j}) \in \mathcal{A}(s_i)$ **do**
    if $e_{i,j}$.status $\neq 200$ **or** $e_{i,j}$.transport_error $\neq \emptyset$ **then**
      Mark transient transport/HTTP failure; $V_{\text{stage}}(s_i) \leftarrow$ False
    else if $\mathcal{V}(e_{i,j}.\text{payload}) = \text{Invalid}$ **or** $e_{i,j}$.choices $= \emptyset$ **then**
      Mark semantic payload defect; $V_{\text{stage}}(s_i) \leftarrow$ False
      {Catches 130 non-exception defects}
    else
      $V_{\text{stage}}(s_i) \leftarrow$ True
      if $j > 1$ **then**
        $R \leftarrow$ True {Valid retry recovery within stage s_i}
      end if
    end if
  end for
end for
if $\exists s_i \in \mathcal{S}$ such that $V_{\text{stage}}(s_i) =$ False **then**
  return Quarantined(InfrastructureFault) {Exclude from denominator}
else
  Evaluate Task Predicate $y \leftarrow \mathcal{P}$(EnvironmentState)
  return Scored(y) {Eligible numeric outcome}
end if

TABLE VII
COMPARISON OF SCORE LEAKAGE ACROSS ACCOUNTING POLICIES UNDER 700 UNRESOLVED FAULTS (2,160 CONTROLLED RUNS).

Accounting Policy	Contaminated Inclusions	Leaked Faults	Interception Rate
Native Benchmark Runners	425 / 700	425	39.3%
Exception-Only Wrapper	130 / 700	130	81.4%
Stage-Aware `evalfault`	**0 / 700**	**0**	**100.0%**

by standard exception wrappers, whereas the remaining 130 cases (empty `choices`, model identity mismatches, and schema-invalid tool arguments returning HTTP 200 OK) bypass exception handlers completely, entering the benchmark score denominator unless intercepted by stage-aware validation. When an upstream provider returns an HTTP 200 status code with semantic payload corruptions, standard client libraries do not raise an exception, treating the returned object as a successful API response and passing the corrupted output into the agent harness where it is tallied as a model failure. `evalfault`'s stage-aware schema validation intercepts these payload defects, proving that structural provenance is necessary.

TABLE VIII
COMPARISON OF ACCOUNTING POLICIES ON 450-CASE SYNTHETIC CHALLENGE PACKET.

Accounting Policy	Invalid Inclusions (out of 216)	False Exclusions (out of
Native Numeric Reporting	216	
Exception-Only Wrapper	72	
Last-Globally-Completed Call	180	
Stage-Aware `evalfault`	**0**	

C. Synthetic Validation on 450 Disjoint Cases (RQ6)

To evaluate whether stage-aware accounting generalizes beyond single provider calls to complex multi-step workflows (concurrency, fallbacks, tool chains), we tested `evalfault` on a disjoint 450-case synthetic validation packet.

A separate, independently implemented raw-evidence checker labeled:

- 216 cases as *invalid*,
- 162 cases as *valid*, and
- 72 cases as *unknown* (ambiguous telemetry).

Table VIII compares four accounting policies across the 450 cases. Native reporting accepted all 216 invalid cases. The exception-only wrapper leaked 72 invalid cases because returned structural defects had no terminal exception. The last-call rule accepted 180 invalid cases because a successful

TABLE IX
DISTRIBUTION OF 129 LIVE PROVIDER REQUESTS AND 31 INTERCEPTED FAULTS ACROSS 50 LIVE TRAJECTORIES.

Live Execution Metric	Trajectory Count	API Request Count
Total Attempted Trajectories	50	129
Reference Valid Trajectories	19	64
Reference Invalid Trajectories	31	65
Breakdown of 31 Intercepted Faults:		
– HTTP 429 Rate Limits / Capacity Shedding	18	49
– HTTP 502 / 503 Gateway Dropouts	9	18
– Premature Socket Teardowns / Timeouts	4	8
Native Score Contamination	**0 / 31 (0.0%)**	–
Guarded Score Contamination	**0 / 31 (0.0%)**	–

unrelated event completed after an unresolved required operation. Stage-aware accounting retained **0 invalid outcomes** and achieved **0 false exclusions** among reference-valid cases. In ambiguous cases, `evalfault` flagged the evidence state as unknown rather than forcing a misleading binary decision.

VI. Exploratory Live Pilot Under Capacity-Shedding Stress (RQ6)

To observe provenance dynamics under realistic, unscripted production conditions, we executed an exploratory live pilot of 50 AgentDojo Workspace trajectories against routed production endpoints (`nemotron-3-super-120b` and `dots-3-note-preview`) involving 129 live API calls.

Methodological Rationale for Routed Stress Endpoints: We deliberately targeted high-stress, dynamically routed provider endpoints for this pilot. Unlike dedicated enterprise capacity, routed shared endpoints experience authentic multi-tenant capacity shedding, aggressive HTTP 429 rate limiting, and intermittent gateway disconnects. This environment serves as an intentional real-world stress testbed to evaluate whether `evalfault`'s provenance boundaries hold under severe provider unreliability.

Table IX details the distribution of live requests and intercepted faults. An independent audit of raw network traces labeled 19 trajectories as valid and 31 as invalid (due to capacity shedding, HTTP 429 rate limits, and gateway drops). Under this bounded setup, all 31 invalid runs halted cleanly before reaching AgentDojo's numeric aggregation function. Consequently, zero invalid runs contaminated the live pilot scores. This honest negative result confirms that live score distortion depends on specific harness exception handling, underscoring the necessity of systematic fault-injection testing.

VII. Discussion, Recommendations, and Limitations

A. Recommendations for Benchmark Authors and Evaluators

Based on our empirical findings across twelve models and six benchmark harnesses, we formulate three actionable principles for LLM agent evaluation:

1) **Mandatory Paired Utility Reporting:** Security benchmarks must never report Attack Success Rate (ASR) in isolation. Every defense must report Benign Utility (BU) and Utility under Attack (UA) on matched task pairs. A defense that reduces ASR while degrading BU is a capability blocker, not a viable defense.
2) **Decouple Attempted from Completed Denominators:** Benchmark leaderboards must explicitly report three separate counts: planned units, attempted executions, and validly scored outcomes. Never convert an infrastructure failure into a zero score.
3) **Enforce Stage-Aware Provenance Guards:** Harnesses should integrate non-invasive provenance observers (such as `evalfault`) to intercept transport, rate-limit, and parser faults at the provider boundary before they enter aggregation functions.

B. Limitations

Our findings are bounded by the evaluated scopes:

- Study 1 evaluated the Workspace suite of AgentDojo; while findings reproduced across ten production models and two open-weight Groq backbones, closed frontier commercial APIs (e.g., GPT-5 or Claude 4.5) may exhibit different capability-restriction thresholds.
- Study 2 evaluated selected native execution paths across six benchmark runners under scripted task boundaries to enable controlled propagation testing; full multi-turn CLI executions may introduce additional confounding factors.
- The synthetic validation packet represents a declared structural grammar of faults rather than an exhaustive model of natural distributed systems failure distributions.

VIII. Conclusion

Valid evaluation of autonomous LLM agents requires that benchmark scores reflect model capabilities rather than behavioral utility destruction or infrastructure flakiness. Our dual-layer audit proves that capability-restriction defenses like Tool Filter create an illusion of security by collapsing benign utility across 100% of evaluated model configurations, while native benchmark runners across six major suites convert 60.7% of unresolved infrastructure faults into false model failure scores. By combining paired security-utility accounting with the `evalfault` stage-aware provenance guard, the community can eliminate denominator contamination and restore measurement integrity to LLM agent benchmarking.

Data and Software Availability

To facilitate full independent replication and community adoption, the complete replication package—including the IEEEtran manuscript sources, raw attempt-level event ledgers across all 4,970 verified executions (2,310 Study 1 completed cells, 2,160 Study 2 controlled executions, 450 synthetic challenge cases, and 50 live pilot trajectories, drawn from 5,296 total planned evaluation units and 9,033+ raw API attempts), the `evalfault` observer harness, and the self-contained automated reproducibility verification suite—is publicly available under the MIT license at: https://github.com/sharif2552/llm-agent-defense-provenance-audit.

REFERENCES

[1] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to benchmark attacks and defenses on LLM-based agents," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 89 481–89 518.

[2] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 14 154–14 175.

[3] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with Spotlighting," in *Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS)*, ser. CEUR Workshop Proceedings, vol. 3920, 2024, pp. 48–62.

[4] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, 2025.

[5] ——, "SecAlign: Defending against prompt injection with preference optimization," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025, arXiv:2410.05451.

[6] Y. Jia, Y. Liu, R. Geng, J. Jia, and N. Z. Gong, "The Task Shield: Enforcing task alignment to defend against indirect prompt injection," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.

[7] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, and F. Tramèr, "CaMeL: Defeating prompt injections by design," *arXiv preprint arXiv:2503.18813*, 2025.

[8] Anthropic Research, "Quantifying infrastructure noise in agentic coding evals," Anthropic Engineering Report, 2024. [Online]. Available: https://www.anthropic.com/research/quantifying-infrastructure-noise-in-agentic-coding-evals

[9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024.

[10] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, G. Wentz, L. Mudrick, B. Parvez, T. Ou, Z. Wang, P. Sharma, D. Ting, R. Dutt, N. Attaluri, A. Cohan, and G. Neubig, "WebArena: A realistic web environment for building autonomous agents," in *International Conference on Learning Representations (ICLR)*, 2024.

[11] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 46 595–46 623.

[12] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 1, pp. 1–49, 2022.

[13] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2014, pp. 643–653.

[14] L. Gao, J. Schulman, and J. Hilton, "Scaling laws for reward model overoptimization," in *International Conference on Machine Learning (ICML)*, 2023, pp. 10 835–10 866.

[15] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *International Conference on Learning Representations (ICLR)*, 2025.

[16] M. Andriushchenko, E. Debenedetti, N. Flammarion, and F. Tramèr, "AgentHarm: A benchmark for measuring harmfulness of LLM agents," in *International Conference on Learning Representations (ICLR)*, 2025.

[17] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, F. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, and R. Pang, "ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.

[18] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan, "τ^2-Bench: Evaluating conversational agents in a dual-control environment," *arXiv preprint arXiv:2506.07982*, 2025.

[19] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AI-Sec)*, 2023, pp. 79–90.

[20] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, and Y. Liu, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, 2024, pp. 3297–3314.

[21] S. Toyer, O. Watkins, E. A. Mendes, J. Svegliato, L. Frank, R. Overfield, C. O'Reilly, S. Bailey, N. Elmaaroufi, R. Toyer, and S. Russell, "Tensor Trust: Interpretable prompt injection attacks from an online game," in *International Conference on Learning Representations (ICLR)*, 2024.

[22] J. Piet, C. Sitawarin, S. Chen, and D. Wagner, "Jailbreak attacks and defenses against large language models: A survey," *IEEE Security & Privacy*, vol. 22, no. 4, pp. 32–44, 2024.

[23] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.

[24] Z. Xiang, Z. Shen, S. Cheng, and Y. Yao, "BadChain: Backdoor attacks against tool-integrated LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 126 581–126 609.

[25] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.

[26] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *International Conference on Learning Representations (ICLR)*, 2024.

[27] X. Liu, Y. Hao, J. She, P. Yuan, T. Gu, M. Ding, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *International Conference on Learning Representations (ICLR)*, 2024.

[28] J. Ye, M. Wu, X. Shen, and Z. Chen, "ToolSword: Uncovering safety hazards in tool-augmented LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 102 871–102 901.

[29] K. Zhu, J. Wang, J. Zhou, Z. Wang, H. Chen, Y. Wang, L. Yang, W. Ye, N. Z. Gong, Y. Zhang, and X. Xie, "PromptBench: Towards evaluating the robustness of large language models on adversarial prompts," *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 10, pp. 5152–5166, 2024.

[30] J. Yi, Y. Xie, F. Zhu, D. Lian, J. Wang, and X. Xie, "Benchmarking and defending against indirect prompt injection attacks on large language models," *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 184–201, 2025.

[31] F. Perez and I. Ribeiro, "Ignore this title and hack this paper: System and user prompts in large language models," *arXiv preprint arXiv:2211.09527*, 2022.

[32] E. Wallace, K. Xiao, J. Leike, L. Weng, and J. Johannes, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," *arXiv preprint arXiv:2404.13208*, 2024.

[33] E. Debenedetti, J. Zhang, M. Balunović, and F. Tramèr, "AgentDojo (version 0.1.35)," GitHub repository, 2024. [Online]. Available: https://github.com/ethz-spylab/agentdojo

[34] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[35] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[36] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[37] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

Md Sharif Hossain received the B.Sc. degree in Computer Science and Engineering from Rajshahi University of Engineering & Technology (RUET), Bangladesh. His research interests include LLM-agent security, empirical benchmark validity, and dependable distributed AI systems.

Faez Mahmud received the B.Sc. degree in Computer Science and Engineering from Rajshahi University of Engineering & Technology (RUET), Bangladesh. His research interests encompass AI safety, autonomous agent architectures, and secure evaluation methodologies for tool-using models.